A Reproducible Benchmarking Framework for Evaluating TLS and mTLS Performance Overhead in Zero Trust-Aligned Apache Kafka Deployments

Nick Efe Oni

Supervisor: Mahshid Delavar

A report submitted in fulfilment of the requirements for the degree of MEng in Software Engineering

in the

School of Computer Science

April 20, 2026

Declaration

All sentences or passages quoted in this report from other people's work have been specifically acknowledged by clear cross-referencing to author, work and page(s). Any illustrations that are not the work of the author of this report have been used with the explicit permission of the originator and are specifically acknowledged. I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this project and the degree examination as a whole.

Name: Nick Efe Oni

Signature: Nick Efe Oni

Date: April 20, 2026

Abstract

an abstract is provided to give a brief overview and summary of the core topics, arguments, and findings . Because it must summarize your entire project—including your final benchmarking results from Chapter 5 and your final conclusions from Chapter 7—you should write the abstract at the very end of the process, once the rest of your dissertation is completely finished.

Contents

1 Introduction **1**

1.1 Background and Motivation . . . 1
1.2 Problem Statement . . . 2
1.3 Research Aim and Objectives . . . 2
1.4 Research Question . . . 3
1.5 Artefact Overview . . . 3
1.6 Dissertation Structure . . . 4

2 Literature Review **5**

2.1 Introduction to Literature Review . . . 5
2.2 Apache Kafka & Distributed Streaming Systems . . . 6
2.3 Performance in Kafka Systems . . . 7
2.4 Security Mechanisms in Kafka . . . 9
2.5 Zero Trust Architecture . . . 10
2.6 Zero Trust in Distributed Streaming Systems . . . 11
2.7 TLS/mTLS Performance Overhead . . . 13
2.8 Benchmarking Methodologies . . . 14
2.9 Synthesis & Research Gap . . . 15

3 Design, Implementation, and Testing **18**

3.1 Chapter Overview . . . 18
3.2 Artefact Requirements . . . 18
3.3 System Architecture . . . 18
3.4 Infrastructure Provisioning . . . 19
3.5 Kafka Cluster Deployment . . . 19
3.6 Security Mode Design . . . 19
3.7 Benchmark Workload Design . . . 20
3.8 Factorial Experiment Design . . . 20
3.9 Execution and Orchestration . . . 20
3.10 Telemetry and Metric Collection . . . 20
3.11 Result Processing and Export . . . 21
3.12 Fault Tolerance and Idempotency . . . 21

3.13 Testing and Validation 21
3.14 Chapter Summary 21

4 Results and Discussion **22**

5 Evaluation, Conclusions, and Further Work **23**

Appendices **26**

A An Appendix of Some Kind **27**

B Ignore - Templatey Stuff **28**
B.1 Section Template Graph 28
B.2 Section Template Comparison 29
B.3 Summary 29

C Another Appendix **30**

List of Figures

2.1 Simplified Apache Kafka architecture showing producer applications publishing events to a cluster of brokers, and consumer applications pulling events from the cluster. 7

2.2 Comparison of Kafka's native zero-copy plaintext path and the TLS/mTLS path. In plaintext mode, Kafka can use `sendfile()` to transfer data efficiently from the page cache to the socket. Under TLS/mTLS, this optimisation is disabled, forcing data through user space for encryption before transmission. 9

2.3 Kafka security data path illustrating how TLS/mTLS, certificate-based identity verification, principal mapping, and authorisation checks are applied across client-broker and broker-to-broker communication. 10

2.4 Conceptual Zero Trust control model for a distributed streaming pipeline, illustrating how mTLS, policy enforcement, least-privilege access, and continuous monitoring are applied across producer, broker, stream-processing, and consumer components within a segmented trust boundary. 13

B.1 A simple figure in LaTeX. Reproduced from http://tinyurl.com/nqtrlj5 with the permission of the copyright owner. 29

List of Tables

3.1 Core artefact requirements 19

B.1 A simple table in LaTeX. 29

Chapter 1

Introduction

1.1 Background and Motivation

Modern distributed systems increasingly rely on event-streaming platforms to connect cloud services, data pipelines, and enterprise applications [1, 2]. Apache Kafka is a widely used open-source event-streaming platform that supports high throughput, low latency, fault tolerance, and horizontal scalability [3–5]. These properties make Kafka a critical component in data-driven systems where large volumes of events must move reliably between producers, brokers, and consumers.

Kafka achieves much of its performance through partition-level parallelism, batching, append-only logs, page-cache utilisation, and zero-copy data transfer [3, 6, 7]. However, these mechanisms also make Kafka sensitive to additional processing in the data path. Security mechanisms are therefore not only deployment choices, as they can affect the communication and computation paths that Kafka relies on for high-throughput streaming.

At the same time, distributed systems face security risks that traditional perimeter-based models do not fully address. In cloud and multi-service environments, internal network access can no longer be treated as inherently trustworthy, as insider threats and lateral movement attacks can originate from within the network boundary [8–10]. This has increased interest in Zero Trust Architecture, whose central principle is commonly summarised as "never trust, always verify" [8–11]. Zero Trust deployments usually emphasise strong identity, mutual authentication, least-privilege access, encrypted communication, monitoring, and policy-based control [9, 11, 12].

Zero Trust Architecture is not a single Kafka feature. It is a broader system-level security paradigm. However, Kafka provides native mechanisms that align with several Zero Trust principles, including transport encryption, certificate-based authentication, broker-to-broker security, SASL authentication, and ACL-based authorisation [7, 13–15]. Among these, TLS and mutual TLS (mTLS) are particularly relevant because they

directly affect Kafka's data path and allow plaintext, server-authenticated, and mutually authenticated transport configurations to be compared.

1.2 Problem Statement

Existing research provides strong foundations for both Zero Trust Architecture and Apache Kafka performance. Zero Trust has been discussed in distributed and streaming systems [8,12,16], while Kafka has been widely analysed as a high-throughput event-streaming platform [4, 5, 17]. However, there is still limited systematic empirical evaluation of the performance impact of Zero Trust-aligned transport security in Kafka-based environments.

This gap matters because TLS and mTLS introduce cryptographic processing into communication paths that Kafka is designed to keep efficient. Kafka's official design documentation states that SSL/TLS prevents the use of the `sendfile` zero-copy optimisation because SSL libraries operate in user space [6]. Wider TLS literature also shows that secure transport can introduce computational and latency overhead, although the scale depends on workload, connection behaviour, and system configuration [18–20]. Therefore, the cost of TLS and mTLS cannot be assumed to be constant across Kafka deployments.

The unresolved problem is the lack of a unified, reproducible, and Kafka-specific framework for measuring this overhead under controlled configurations. In particular, there is limited evidence showing how plaintext, TLS, and mTLS compare when workload and deployment parameters are matched, and how this overhead changes across Kafka variables such as broker count, replication factor, acknowledgements, batching, compression, producer concurrency, and message size.

1.3 Research Aim and Objectives

The aim of this dissertation is to design, implement, and evaluate a reproducible benchmarking framework for quantifying the performance overhead introduced by TLS and mTLS in high-throughput Apache Kafka deployments.

The objectives are:

1. To analyse Kafka's performance mechanisms and identify how transport security may affect them.
2. To design and implement a cloud-based Kafka benchmarking artefact supporting plaintext, TLS, and mTLS.
3. To provision reproducible Kafka deployments using infrastructure-as-code and automated orchestration.

4. To execute matched benchmark workloads across security modes using explicit configuration files.
5. To measure throughput and latency for each benchmark run.
6. To collect host-level telemetry from Kafka brokers and benchmark clients, including CPU, memory, disk, and network activity.
7. To evaluate how TLS and mTLS overhead varies across Kafka workload and deployment parameters.
8. To include targeted consumer-side validation to assess whether secure transport also affects Kafka's read path.

1.4 Research Question

This dissertation addresses the following research question:

> "What is the performance overhead introduced by Zero Trust-aligned security mechanisms, particularly mTLS, in high-throughput Kafka-based distributed systems?"

1.5 Artefact Overview

To answer this research question, this dissertation designs, implements, and evaluates a configurable Kafka benchmarking artefact. The artefact provisions Apache Kafka clusters on AWS EC2 using Terraform, deploys Kafka in KRaft mode, and supports three transport-security configurations: plaintext, TLS, and mTLS. Benchmark execution is driven by structured configuration files so that equivalent workloads can be repeated across security modes and compared under matched conditions.

The main evaluation uses a producer-side factorial benchmark campaign. This campaign varies broker count, replication factor, minimum in-sync replicas, message size, target throughput, producer count, acknowledgement mode, batch size, and compression type. This allows the artefact to examine whether TLS and mTLS introduce overhead, and how that overhead changes under different Kafka workload and durability settings.

A smaller targeted consumer-side validation study is also included. This does not reproduce the full producer factorial campaign. Instead, it evaluates whether secure transport also affects Kafka's read path under selected workloads, especially where mTLS processing interacts with larger messages and compressed payloads.

For each benchmark run, the artefact records application-level metrics, including throughput

and latency, alongside host-level telemetry from the benchmark client and Kafka brokers. This includes CPU utilisation, memory utilisation, disk activity, and network activity. Raw benchmark logs are parsed into structured JSON and CSV outputs, allowing reproducible comparison of plaintext, TLS, and mTLS results.

1.6 Dissertation Structure

The remainder of this dissertation is organised as follows. Chapter 2 reviews the literature on Apache Kafka, Kafka performance, Kafka security, Zero Trust Architecture, TLS/mTLS overhead, and benchmarking methodology. It identifies the research gap addressed by this dissertation.

Chapter 3 describes the design, implementation, and testing of the Kafka benchmarking artefact. It covers the infrastructure design, Kafka deployment model, security-mode configuration, factorial workload generation, orchestration scripts, telemetry collection, result-processing pipeline, and validation strategy.

Chapter 4 presents and discusses the benchmark results. It compares plaintext, TLS, and mTLS under matched workloads and analyses the observed throughput, latency, and host-telemetry behaviour.

Chapter 5 evaluates the dissertation objectives, summarises the main findings, discusses limitations, and identifies directions for future work.

Chapter 2

Literature Review

2.1 Introduction to Literature Review

This literature review examines the areas needed to understand the performance impact of Zero Trust-aligned transport security in Apache Kafka. It does not treat Kafka and Zero Trust as separate topics. Instead, it focuses on the interaction between Kafka's high-throughput architecture and security mechanisms such as TLS and mutual TLS (mTLS), which add cryptographic processing to Kafka's communication paths.

The chapter first reviews Apache Kafka as a distributed event-streaming platform, including its producer, broker, consumer, topic, partition, and replication model. It then analyses the mechanisms that contribute to Kafka's performance, such as append-only logs, batching, compression, page-cache utilisation, partition-level parallelism, pull-based consumption, and zero-copy data transfer. These mechanisms are important because they form the baseline against which TLS and mTLS overhead must be measured.

The review then examines Kafka's native security mechanisms, including encryption, authentication, and authorisation. This leads into a discussion of Zero Trust Architecture, where a key distinction is made: Zero Trust is a system-level security paradigm, while Kafka only provides mechanisms that can support Zero Trust-aligned deployments. In this dissertation, TLS and mTLS are therefore evaluated as measurable transport-security mechanisms aligned with Zero Trust principles, rather than as a complete implementation of Zero Trust Architecture.

The chapter then reviews existing work on TLS and mTLS performance overhead, followed by benchmarking methodologies for Kafka and distributed systems. This is necessary because existing TLS studies often focus on web servers or generic client-server systems, while Kafka has a replicated, partitioned, and batch-oriented data path. Finally, the chapter synthesises these areas into the research gap addressed by this dissertation: the lack of a reproducible and Kafka-specific empirical evaluation of plaintext, TLS, and mTLS

under matched high-throughput configurations.

2.2 Apache Kafka & Distributed Streaming Systems

Distributed streaming systems, such as Apache Kafka, are designed to store, process, and move continuous streams of data across distributed applications [3, 4, 6]. Kafka is widely used because it provides fault tolerance, high availability, reliability, and high-throughput event delivery [3, 4, 17]. These properties make it suitable for cloud and enterprise systems where many applications must exchange data reliably and at scale.

Apache Kafka is built around a cluster of servers called brokers and client applications called producers and consumers [3, 4]. Producers publish events to Kafka topics, while consumers subscribe to and read those events from the brokers [3, 4]. Brokers therefore act as the intermediary layer between data-producing and data-consuming applications. A simplified version of this architecture is shown in Figure 2.1.

Kafka stores events in topics, which are implemented as append-only, time-ordered logs [3, 6, 17]. Topics are divided into partitions, and these partitions can be distributed across multiple brokers. This allows Kafka to scale horizontally because different brokers and clients can process different partitions in parallel [3, 4, 6]. Replication further improves availability and durability by storing copies of partition data across multiple brokers [3, 6].

Kafka also uses a pull-based consumption model. Instead of brokers pushing events to consumers, consumers fetch records at their own rate [6]. This decouples producers from consumers and allows consumers to process large batches without being directly overwhelmed by the production rate. This model contributes to Kafka's scalability and fault tolerance, but it also means that performance depends heavily on client behaviour, batching, partitioning, and broker configuration.

These design choices create important trade-offs. Kafka performance is affected by acknowledgement settings, replication, batching, partitions, broker configuration, and client configuration [6, 7, 13, 14, 17]. For example, stronger durability settings can increase latency because producers may need to wait for more replicas to acknowledge a write. Similarly, poor partitioning or unsuitable broker configuration can limit throughput even when the underlying platform is capable of higher performance.

Therefore, Kafka performance should not be treated as a fixed property of the platform. It is shaped by the selected configuration, deployment environment, workload, and coordination costs [6,7,13,14,17]. This is important for this dissertation because security mechanisms such as TLS and mTLS add communication and cryptographic processing to Kafka's distributed data path. Their overhead must therefore be evaluated within Kafka's architecture, rather

than treated as a generic network cost [7, 13, 14, 18].

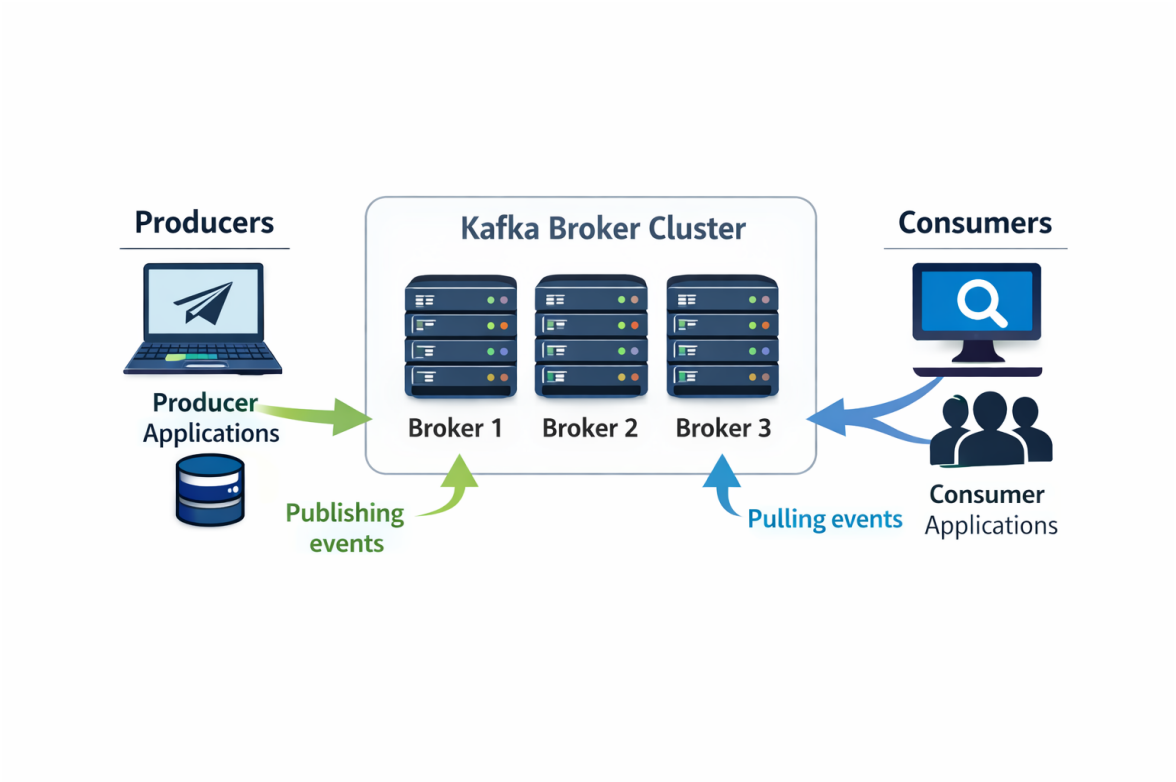

Figure 2.1: Simplified Apache Kafka architecture showing producer applications publishing events to a cluster of brokers, and consumer applications pulling events from the cluster.

2.3 Performance in Kafka Systems

Kafka performance comes from several architectural choices that reduce unnecessary data movement and allow data to be processed in parallel. Instead of relying on random-access data structures such as B-Trees, Kafka stores records in append-only logs, allowing efficient sequential writes [3, 6, 17]. Kafka also relies heavily on the operating system page cache and uses zero-copy I/O, specifically the `sendfile` system call, to transfer data from the page cache to the network socket with minimal copying [6]. These optimisations make Kafka fast, but they also make it sensitive to mechanisms that move processing back into user space, such as TLS and mTLS [6, 13, 18, 19]. This contrast is shown in Figure 2.2.

Kafka also gains performance through partitioning, batching, compression, and pull-based consumption [3, 4, 6, 17]. Topics are divided into partitions, which can be distributed

across brokers and accessed in parallel by producers and consumers [3, 4, 6]. Producers can group records into larger batches, reducing network round-trip overhead and improving throughput [6, 7, 17]. Consumers then fetch records using a pull-based model, allowing them to read batches at their own processing rate [4–6, 14].

Compression further changes Kafka's performance behaviour. By compressing batches, producers can reduce network usage and broker storage pressure [6, 7, 14]. However, this also shifts some computational cost to producers and consumers, especially during compression and decompression. This matters for this dissertation because TLS and mTLS also introduce cryptographic processing. Therefore, the combined effect of compression and transport security may depend on CPU availability, message size, and client concurrency.

Kafka configuration choices also create explicit throughput-latency-durability trade-offs [6, 7, 13, 14, 17]. For example, producers can use different acknowledgement settings. Waiting for all in-sync replicas improves durability, but can increase latency compared with weaker acknowledgement settings [6, 7, 13]. Similarly, replication factor, minimum in-sync replicas, batch size, and producer concurrency can all change the balance between throughput, latency, and reliability. This means Kafka performance is not a fixed property of the platform, but an outcome of workload and configuration choices.

This relationship can also be understood using a simple queueing-theoretic model. In publish-subscribe systems, average latency (L) can be expressed in terms of processing time (τ) and system utilisation (U) as:

$$L = \frac{\tau}{1 - U} \tag{2.1}$$

[4, 5, 19]. As processing time increases, utilisation moves closer to saturation, and latency can grow non-linearly rather than linearly [4, 5, 17, 19]. This is important because TLS and mTLS can increase processing time by adding encryption, decryption, and certificate-based authentication to Kafka communication paths [6, 13, 18, 20].

Therefore, Kafka performance must be evaluated under controlled configurations rather than assumed from general platform claims. The impact of transport security may vary with batching, compression, producer count, replication, acknowledgement mode, message size, and broker resources. This motivates the need to examine Kafka's native security mechanisms before measuring their performance impact in the artefact.

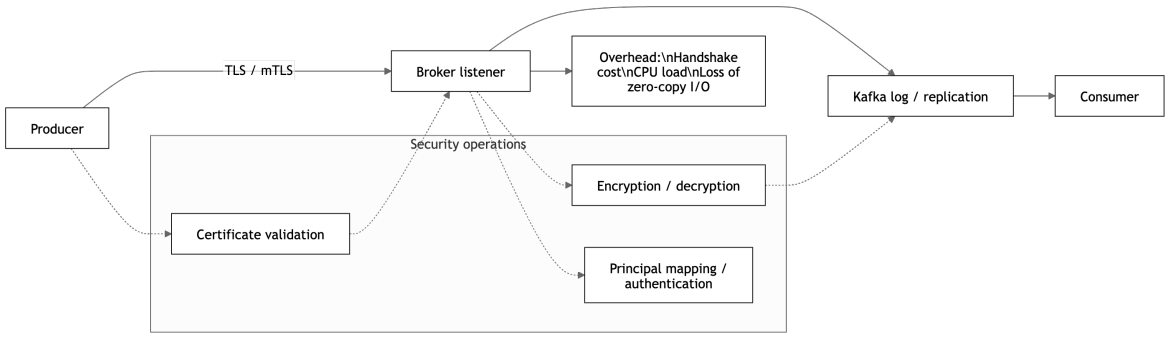

Figure 2.2: Comparison of Kafka's native zero-copy plaintext path and the TLS/mTLS path. In plaintext mode, Kafka can use `sendfile()` to transfer data efficiently from the page cache to the socket. Under TLS/mTLS, this optimisation is disabled, forcing data through user space for encryption before transmission.

2.4 Security Mechanisms in Kafka

Kafka provides native security mechanisms for protecting data streams, including encryption, authentication, and authorisation [3, 7, 13–15]. These mechanisms are important because Kafka is often deployed across distributed services where producers, brokers, and consumers exchange sensitive data. However, enabling security can also change Kafka's data path by adding cryptographic processing, identity verification, and access-control checks [6, 7, 13, 14].

Encryption in transit is provided through Transport Layer Security (TLS). TLS protects data moving between Kafka clients and brokers, and can also be configured for broker-to-broker communication [7, 13, 14]. Kafka exposes this through listener and SSL configuration options, including `ssl.enabled.protocols` and `ssl.cipher.suites`, which allow operators to control the supported TLS protocols and cipher suites [7, 13, 14]. This provides confidentiality and protection against eavesdropping or man-in-the-middle attacks, but it also introduces encryption and decryption work into Kafka's communication path.

This matters for performance because Kafka's plaintext path can use efficient operating-system mechanisms such as `sendfile`. Kafka's design documentation states that this zero-copy optimisation is not used when SSL is enabled because SSL libraries operate in user space [6]. Therefore, TLS does not only add cryptographic cost; it can also change how data moves between disk, memory, and the network. This makes Kafka-specific measurement necessary, because generic TLS overhead results from web servers or simple client-server systems do not fully represent Kafka's replicated and batch-oriented architecture [17–19].

Kafka can also support mutual TLS (mTLS). In this mode, clients present certificates

to brokers, and brokers verify those identities using Kafka SSL configuration such as `ssl.client.auth` [7, 13, 14]. This allows producers, consumers, and brokers to authenticate using cryptographically signed X.509 certificates. A simplified view of this security path is shown in Figure 2.3.

mTLS is especially relevant to Zero Trust-aligned deployments because it provides encrypted communication and certificate-based identity verification. However, it can also increase the amount of cryptographic work in the system. In Kafka, this work may occur not only on client-broker communication, but also on broker-to-broker traffic when internal replication is secured [6, 13, 20]. Reconnection events can also create additional TLS or mTLS handshake load, especially when many clients reconnect at the same time [13,18,19].

Therefore, Kafka's native security mechanisms create a direct performance-security trade-off. They improve confidentiality, authentication, and access control, but they may also increase CPU usage, latency, and data movement through user space. For this reason, TLS and mTLS are the main focus of this dissertation. They are measurable transport-security mechanisms that align with Zero Trust principles, while also directly affecting the Kafka data path that enables high throughput.

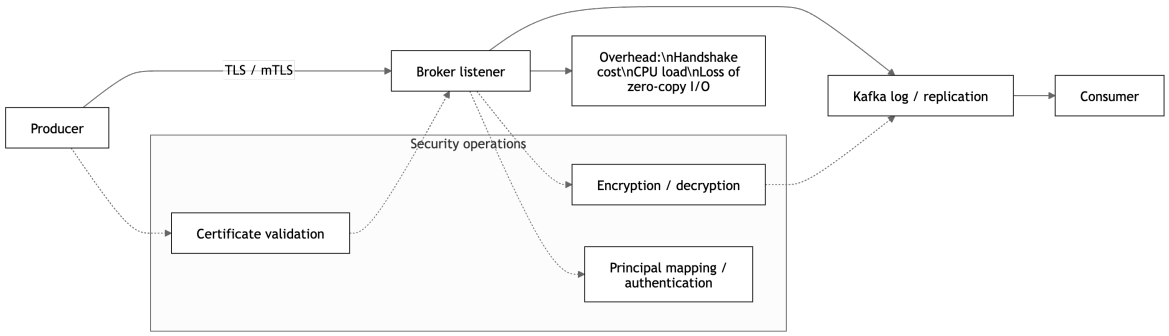

Figure 2.3: Kafka security data path illustrating how TLS/mTLS, certificate-based identity verification, principal mapping, and authorisation checks are applied across client-broker and broker-to-broker communication.

2.5 Zero Trust Architecture

Zero Trust Architecture (ZTA) is a security model developed in response to the limits of traditional perimeter-based security [8–10, 12]. Traditional models often assume that systems inside the network boundary are trustworthy. ZTA rejects this assumption and instead follows the principle of "never trust, always verify" [8–11]. In this model, access is

continuously evaluated using identity, context, policy, and device or service state, rather than being granted simply because a request originates from inside the network.

The core principles of ZTA include strong identity, least-privilege access, continuous verification, encrypted communication, monitoring, and fine-grained policy enforcement [8–11]. These principles are commonly implemented through architectural components such as Policy Decision Points (PDPs) and Policy Enforcement Points (PEPs), which separate policy decisions from policy enforcement [8, 9, 11]. ZTA is therefore not a single tool or protocol, but a system-level security approach.

Cryptographic transport security is one part of this wider model. TLS and mTLS are often used to protect communication channels and verify service identities in distributed systems [8–10, 20]. TLS provides encrypted communication and server authentication, while mTLS extends this by requiring both sides of a connection to authenticate with certificates. This makes mTLS especially relevant to Zero Trust-aligned machine-to-machine communication, where services should not be trusted purely because they are inside the same network.

However, ZTA also introduces operational and performance challenges. Implementing it can require changes to identity management, certificate handling, monitoring, network segmentation, and access-control policy design [8–12]. In high-throughput systems, these controls can add processing and coordination overhead. This creates the key tension for this dissertation: the same mechanisms that improve trust and identity assurance may also affect the latency and throughput of systems such as Apache Kafka [6, 18, 19].

2.6 Zero Trust in Distributed Streaming Systems

The challenges of Zero Trust Architecture also apply to distributed streaming systems [1, 8–10, 12]. Modern event-streaming pipelines often span cloud services, remote devices, multi-cloud environments, and third-party networks. In these settings, components cannot be assumed to be trustworthy simply because they are inside the same network. Each producer, broker, stream processor, and consumer may need to be authenticated, authorised, encrypted, and monitored [8–10, 12]. Figure 2.4 illustrates this type of control model.

This creates a direct tension for streaming systems. These systems are designed for continuous data movement, low latency, and high throughput. However, Zero Trust-aligned controls such as mutual authentication, encryption, segmentation, and policy enforcement add work to the same communication paths that carry the data [12, 18–20]. Therefore, the main issue is not whether these controls are useful, but how their cost changes under realistic streaming workloads.

Access control is one example of this tension. Role-Based Access Control (RBAC) is widely used because it is simple to understand and operate, but it can lack the granularity required in dynamic distributed systems [8–10]. Attribute-Based Access Control (ABAC) provides finer-grained control by evaluating attributes such as identity, device state, time, location, and environmental context [8, 9, 12]. However, ABAC can be harder to configure correctly, and run-time policy evaluation can add latency if it is placed on the critical data path [8–10, 12].

Transport security creates a similar issue. Some studies suggest that modern TLS can have low steady-state overhead when connections are persistent and hardware is adequately provisioned [15, 19]. However, this does not fully resolve the problem for streaming systems. Event-streaming workloads may involve large message volumes, many clients, broker-to-broker communication, compressed batches, and reconnect events. In these cases, TLS and mTLS overhead may depend on workload shape, connection behaviour, certificate verification, and available CPU resources [12, 18–20].

This is particularly important for Apache Kafka because Kafka is designed around efficient batching, partitioning, replication, and zero-copy data transfer [6, 17]. Zero Trust-aligned transport controls, especially TLS and mTLS, can interact with these mechanisms by moving work into user space and adding cryptographic processing to client-broker and broker-to-broker communication. Therefore, the performance impact cannot be inferred from generic TLS studies alone. It must be measured in Kafka under controlled and reproducible configurations.

This establishes the need for the empirical approach used in this dissertation. The relevant question is not whether Zero Trust principles are valuable, but what performance cost is introduced when selected Zero Trust-aligned mechanisms are applied to a high-throughput streaming platform. This motivates the comparison of plaintext, TLS, and mTLS Kafka deployments under matched workloads.

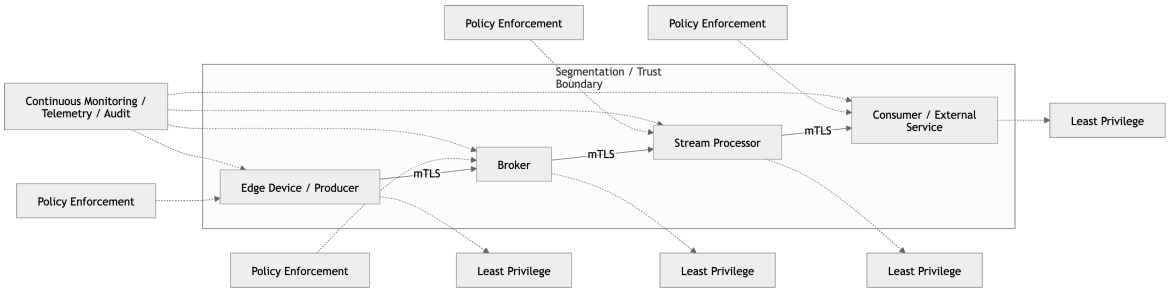

Figure 2.4: Conceptual Zero Trust control model for a distributed streaming pipeline, illustrating how mTLS, policy enforcement, least-privilege access, and continuous monitoring are applied across producer, broker, stream-processing, and consumer components within a segmented trust boundary.

2.7 TLS/mTLS Performance Overhead

Existing TLS performance studies show that encrypted transport can introduce measurable computational overhead [18,19]. Coarfa et al. separate this cost into connection setup, which uses asymmetric cryptography, and bulk data transfer, which uses symmetric encryption [18]. Their web-server experiments report a throughput degradation factor between 3.4 and 9 compared with plaintext baselines, depending on workload and session reuse [18]. They also identify RSA-based operations as a major source of overhead, consuming between 13% and 58% of web-server processing time under the evaluated workloads [18].

However, these findings cannot be transferred directly to Apache Kafka. Traditional web-server TLS benchmarks often model short-lived request-response workloads, where new connections are frequently created and where data is already handled in user space [18, 19]. Kafka is different. It is based on long-lived client-broker connections, batching, partitions, replication, and operating-system optimisations such as page-cache reuse and zero-copy I/O [6, 7, 14]. Kafka's design documentation states that `sendfile` is not used when SSL is enabled because SSL libraries operate in user space [6]. This means TLS affects Kafka not only through encryption cost, but also by changing the data-movement path.

Persistent-connection studies give a different view of TLS overhead. Iorio et al. show that persistent encrypted channels can reduce repeated handshake costs, allow TCP behaviour to stabilise, and make steady-state TLS latency small on sufficiently provisioned devices [19]. This is important because Kafka also uses long-lived connections. However, these studies still do not fully represent Kafka because they focus on generic client-server communication rather than replicated, broker-mediated streaming systems [6, 19].

In Kafka, TLS and mTLS may apply across several communication paths. These include

producer-to-broker, consumer-to-broker, and broker-to-broker communication [3, 6, 13]. When mTLS is enabled, certificate-based authentication can also be required in addition to encryption [13, 20]. Therefore, the cost is not limited to a single client-server channel. It may interact with replication, batching, compression, partitioning, and broker resources [6, 7, 13, 14].

The Kafka benchmarking literature also shows a reproducibility problem. Mohammad reports that although many Kafka studies include empirical benchmarks, they often omit important configuration details such as partition count, replication factor, batch size, linger time, compression settings, and workload characteristics [17]. This makes it difficult to compare results across studies, because observed performance differences may come from configuration changes rather than security overhead alone [7, 13, 17].

Therefore, the performance impact of TLS and mTLS in Kafka should be treated as configuration-dependent. It depends on how cryptographic work is distributed across clients and brokers, how it interacts with batching and compression, and how close the system is to CPU, network, or disk saturation [6, 7, 13, 18]. This motivates the need for a controlled benchmarking framework that compares plaintext, TLS, and mTLS under matched Kafka configurations.

2.8 Benchmarking Methodologies

Performance evaluation in distributed systems often starts with theoretical models. Network studies may use topological metrics such as bisection bandwidth or sparsest cut to estimate throughput limits [21]. Kafka studies may also use queueing models to relate latency to processing time and utilisation [5]. These models are useful because they explain why saturation can increase latency and reduce throughput. However, they are not sufficient on their own.

Jyothi et al. show that cut-based metrics do not always predict worst-case throughput, since a topology with a stronger cut metric can still perform worse under certain traffic patterns [21]. Similarly, Kafka producer evaluations show that queueing models can overestimate performance because they do not fully capture operating-system overhead, thread coordination, CPU contention, and network I/O contention [5]. Therefore, theoretical models are useful for framing the problem, but empirical benchmarking is required to measure the real behaviour of Kafka under load [6, 17].

Latency measurement also needs care. Some empirical studies rely on network-level round-trip time (RTT), using tools such as ping, traceroute, or transport-level measurements [19]. These metrics can show whether the network path is stable, but they do not capture the full application-level path. Iorio et al. argue that application-level

Flow Completion Time (FCT) is more representative because it includes protocol processing, endpoint behaviour, buffering, proxying, and other effects that network-level probes can miss [19].

This matters because distributed systems can appear stable at median latency while still showing large tail-latency increases. Iorio et al. show that performance can remain similar up to high percentiles but diverge at the extreme tail due to shared infrastructure, endpoint processing, buffering, and transport behaviour [19]. Therefore, Kafka benchmarking should not rely only on average or median latency. It should also capture maximum latency and, where available, high-percentile behaviour.

Kafka benchmarking also suffers from reproducibility issues. Researchers use standardised suites such as TPCx-Kafka, synthetic frameworks such as the Yahoo Streaming Benchmark, and custom workload generators [17]. Standardised suites provide recognisable baselines, while custom workloads can model more domain-specific pipelines such as finance, IoT, retail, or machine-learning systems [12, 15, 17]. However, Mohammad's review of 42 Kafka event-streaming studies shows that benchmark reporting is often incomplete and inconsistent [17].

This lack of transparency is a major problem for security-overhead evaluation. Kafka performance depends on parameters such as partition count, replication factor, message size, batch size, compression, producer concurrency, consumer lag, and acknowledgement settings [7, 13, 17]. If these are not reported or controlled, then performance differences may be caused by configuration variation rather than TLS or mTLS overhead. Sundbom also shows that aggregate metrics can hide internal client-side effects, such as producer queueing and outbuffer contention [5].

Therefore, evaluating Kafka transport-security overhead requires a transparent and reproducible benchmarking method. The benchmark must use explicit configuration files, matched workloads, structured output, and enough metric detail to connect application-level behaviour with resource-level causes. This motivates the artefact developed in this dissertation, which uses controlled factorial benchmark plans to compare plaintext, TLS, and mTLS under matched Kafka configurations.

2.9 Synthesis & Research Gap

The literature shows that Apache Kafka is a high-throughput distributed event-streaming platform whose performance depends on several connected mechanisms. These include append-only logs, page-cache utilisation, batching, partition-level parallelism, pull-based consumption, compression, replication, and zero-copy data transfer [3, 6, 7, 14]. These mechanisms allow Kafka to move large volumes of data efficiently, but they also make

performance sensitive to configuration choices and additional processing in the data path [5–7, 13, 17]. Therefore, Kafka performance should be treated as a configurable outcome, not as a fixed platform property.

The literature also shows that modern distributed systems increasingly require security models that do not rely on implicit trust inside the network [8–10]. Zero Trust Architecture addresses this by emphasising strong identity, least-privilege access, continuous verification, encrypted communication, monitoring, and fine-grained control [8–10]. However, Zero Trust is not a single feature that can be enabled in Kafka. Instead, Kafka provides mechanisms that can support Zero Trust-aligned deployments, including TLS encryption, mTLS-based authentication, SASL authentication, ACL authorisation, and broker-to-broker security configuration [7, 13, 14].

Among these mechanisms, TLS and mTLS are the most relevant to this dissertation because they directly affect Kafka's data path. They provide a measurable basis for comparing plaintext, server-authenticated, and mutually authenticated transport configurations [6, 13]. This creates a performance-security tension. Kafka's plaintext path can rely on efficient operating-system mechanisms such as `sendfile`, but Kafka's design documentation states that this optimisation is not used when SSL is enabled because SSL libraries operate in user space [6]. TLS and mTLS can therefore introduce both cryptographic processing and additional user-space data movement.

Existing TLS studies provide useful evidence, but they do not fully answer the Kafka-specific problem. Web-server studies show that TLS can introduce measurable overhead, especially during connection setup and asymmetric cryptographic operations [18, 20]. Persistent-connection studies show that this overhead can be smaller once handshake costs are amortised [19]. However, Kafka is not a stateless web server or a single client-server channel. It is a replicated, partitioned, batch-oriented distributed log with producer-broker, consumer-broker, and broker-to-broker communication paths [3, 6, 13]. Therefore, Kafka-specific measurement is required.

Kafka performance studies also do not fully resolve the problem. Existing work often examines throughput, batching, replication, producer parallelism, or benchmark-suite design without isolating TLS, mTLS, client authentication, and encrypted broker communication under matched configurations [5, 13, 17]. Where Kafka security is discussed, evaluations are often domain-specific, configuration-limited, or insufficiently reproducible [15, 17]. This limits the ability to compare results across workloads and deployment conditions.

The benchmarking literature further strengthens this gap. Kafka performance depends on variables such as broker count, replication factor, producer concurrency, batch size, acknowledgement mode, compression, message size, and partitioning [6, 7, 13]. If these

parameters are not controlled or reported, observed differences may come from configuration variation rather than security overhead. Coarse network metrics and theoretical models are also insufficient on their own because they do not fully capture operating-system overhead, thread coordination, network I/O contention, or application-level latency [5, 19, 21]. Therefore, evaluating Kafka security overhead requires transparent, repeatable, and configuration-explicit benchmarking.

The research gap addressed by this dissertation is therefore the lack of a reproducible, systematic, and Kafka-specific benchmarking framework for quantifying the performance overhead of Zero Trust-aligned transport-security mechanisms, particularly TLS and mTLS, under controlled high-throughput configurations [6, 13, 17]. This dissertation addresses that gap by designing and evaluating a configurable Kafka benchmarking artefact that provisions cloud-based Kafka clusters, executes matched plaintext, TLS, and mTLS workloads, records producer-side throughput and latency, captures host-level telemetry from clients and brokers, and includes targeted consumer-side validation.

The contribution is both methodological and empirical. Methodologically, the artefact provides a reproducible approach for evaluating Kafka transport-security overhead under matched conditions. Empirically, it provides evidence about how TLS and mTLS affect throughput, latency, and resource utilisation across Kafka workload and deployment parameters.

Chapter 3

Design, Implementation, and Testing

3.1 Chapter Overview

Purpose: explain that Chapter 3 describes the artefact used to answer the research question.

Example wording:This chapter describes the design, implementation, and testing of the Kafka benchmarking artefact developed to evaluate the performance overhead of Zero Trust-aligned transport-security mechanisms. The chapter first derives the artefact requirements from the research question, then presents the system architecture, infrastructure provisioning approach, Kafka deployment model, benchmark execution framework, telemetry pipeline, result-processing workflow, and testing strategy.

3.2 Artefact Requirements

Split into functional and non-functional requirements.

Functional requirements:

Provision Kafka infrastructure. Deploy plaintext, TLS, and mTLS configurations. Run matched producer workloads. Run targeted consumer validation workloads. Capture throughput and latency. Capture host telemetry. Export structured results. Non-functional requirements:

Reproducibility. Resumability. Fault tolerance. Configuration transparency. Cost control. Comparability across security modes. Example table:

3.3 System Architecture

This needs a diagram.

Recommended diagram: Local workstation — — Terraform + orchestration scripts v AWS VPC — +– Kafka broker 1 +– Kafka broker 2 +– Kafka broker 3 +– Kafka broker 4

Requirement	Description
Reproducibility	Benchmark runs must be generated from explicit configuration files and produce structured outputs.
Comparability	Plaintext, TLS, and mTLS runs must use matched workload and deployment parameters.
Resumability	Long-running campaigns must recover from interruption without rerunning completed rows.
Telemetry	Each run must capture application metrics and host-level resource utilisation.

Table 3.1: Core artefact requirements

+– Kafka broker 5 +– Benchmark client Explain:

Terraform provisions AWS EC2 instances. Kafka traffic uses private IPs. SSH orchestration uses public IPs. Benchmark client runs producer/consumer tools. Results are copied back locally.

3.4 Infrastructure Provisioning

Discuss:

Terraform. AWS EC2. VPC/subnet/security groups. Broker count phases: 3 and 5 brokers. Benchmark client instance. 40 GB gp3 root volumes. Why VMs were used instead of containers. Important argument: Virtual machines were selected over containers because the experiment required control over machine-level resource behaviour and host telemetry. This allowed CPU, memory, network, and disk activity to be measured in a way that aligned with the dissertation's performance model.

3.5 Kafka Cluster Deployment

Cover:

Kafka KRaft mode. Brokers. Controller quorum. Topic creation. Partitions. Replication factor. min.insync.replicas. Readiness checks. Mention that readiness is not just systemd: Broker readiness was validated using Kafka API checks rather than relying only on systemd process status. This reduced the risk of starting benchmark runs against brokers that were running at the process level but not yet able to serve Kafka requests.

3.6 Security Mode Design

Three modes:

Mode Description Plaintext Baseline with no transport encryption TLS Server-authenticated encrypted transport mTLS Encrypted transport with client certificate authentication Important wording: These modes do not represent a complete Zero Trust

Architecture. Instead, TLS and mTLS are evaluated as Zero Trust-aligned transport-security mechanisms because they provide encrypted communication and certificate-based identity verification. Mention current protocol configuration carefully: The TLS and mTLS configurations permit TLS 1.3 and TLS 1.2, with protocol negotiation handled by the Kafka/JDK SSL stack. Unless you later force TLS 1.3 only.

3.7 Benchmark Workload Design

Producer campaign:

Main evaluation. Kafka kafka-producer-perf-test.sh. Concurrent producer support. Parameters: message$_{s}ize_{b}ytestarget_{m}essages_{p}er_{s}econdbatch_{s}izeacksproducer_{c}ountcompression_{t}ypereplicatio$

Targeted validation slice. Not full factorial. Measures read-path behaviour. Includes: plaintext/TLS/mTLS message size compression consumer count trials Phrase carefully: The consumer-side benchmark was intentionally designed as a targeted validation slice rather than a full factorial campaign. This kept the evaluation tractable while still testing whether secure transport also affects Kafka's read path.

3.8 Factorial Experiment Design

Explain:

JSON config files. JSONL generated plan. Deterministic run IDs. Validity constraints. Matched security-mode comparison. Mention final campaign: 5,184 producer rows total 1,728 plaintext 1,728 TLS 1,728 mTLS But avoid saying all are complete unless they are. Use: The final producer campaign design contains 5,184 planned rows after validity constraints are applied. Not: The final producer campaign executed 5,184 rows.

3.9 Execution and Orchestration

Explain scripts:

generate$_{f}actorial_{p}lan.shrun_{f}actorial_{p}lan.shparserscriptsaggregationscriptsexportscriptsKeydetails$:

started.jsonl completed.jsonl failures.jsonl checkpoint file summary.json summary.csv

3.10 Telemetry and Metric Collection

Metrics:

throughput records/sec throughput MB/sec avg latency max latency CPU utilisation memory utilisation network RX/TX disk read/write sectors telemetry host count workload metadata failure count Important caveat: The interval-derived p95 and p99 fields are treated as diagnostic indicators rather than true per-record latency percentiles. This protects you from overclaiming.

3.11 Result Processing and Export

Explain:

raw logs metadata.json result.json summary.json summary.csv comparison exports plots/tables Mention dissertation-ready outputs:

CSV tables. LaTeX tables. SVG plots. security comparison exports.

3.12 Fault Tolerance and Idempotency

This is a very important section for marks.

Mention:

deployment checkpoints; benchmark checkpoints; atomic writes; unique run IDs; topic cleanup; per-run topic deletion; result copying through temporary directories; retry wrappers; API readiness checks; no rerun of completed rows. This shows "precise and methodical design".

3.13 Testing and Validation

Separate testing types:

Test Type Purpose Terraform validation Confirm infrastructure can be provisioned Broker readiness checks Confirm Kafka API availability Plaintext smoke Validate baseline producer path TLS smoke Validate encrypted transport mTLS smoke Validate client-certificate authentication Consumer smoke Validate read-path benchmark Resume test Confirm checkpointed execution skips completed rows Heavy-row test Confirm large messages/concurrent producers work Parser test Confirm raw logs become structured JSON/CSV Use current evidence:

plaintext historical validation; TLS smoke; mTLS smoke; mTLS consumer full slice complete; mTLS producer phase in progress; 0 failures so far in completed current mTLS rows. But keep final counts in Chapter 4 if they are still changing.

3.14 Chapter Summary

End with: This chapter has described the artefact design and implementation used to evaluate Kafka transport-security overhead. The system combines cloud-based Kafka deployment, security-mode configuration, factorial workload generation, host telemetry, resumable orchestration, and structured result processing. The following chapter uses the outputs of this artefact to evaluate the performance overhead of TLS and mTLS under matched Kafka workloads.

Chapter 4

Results and Discussion

This is where you present your findings (e.g., your throughput and latency data). To reach the 70+ band, your results must be presented in a logical order alongside a "perceptive and critical discussion" of what the data actually means

Chapter 5

Evaluation, Conclusions, and Further Work

Your final chapter must evaluate how well you satisfied your original objectives, provide an insightful summary of your conclusions, and discuss appropriate directions for future research

Bibliography

[1] A. Kumar, "Distributed systems in modern enterprise architecture: Challenges and solutions," *IJSAT*, vol. 16, no. 1, p. 2447, 2025.

[2] "Powered by," https://kafka.apache.org/powered-by/, accessed: 2026-03-24.

[3] "Introduction," https://kafka.apache.org/42/getting-started/introduction/, accessed: 2026-03-24.

[4] T. P. Raptis and A. Passarella, "A survey on networked data streaming with apache kafka," *IEEE Access*, vol. 11, pp. 85 333–85 350, 2023.

[5] A. Sundbom, "Parallel kafka producer applications: Their performance and its limitations," Master's thesis, KTH Royal Institute of Technology, 2023.

[6] "Design," https://kafka.apache.org/42/design/design/, accessed: 2026-03-24.

[7] "Producer configs," https://kafka.apache.org/42/configuration/producer-configs/, accessed: 2026-03-26.

[8] H. Kang, G. Liu, Q. Wang, L. Meng, and J. Liu, "Theory and application of zero trust security: A brief survey," *Entropy*, vol. 25, no. 12, p. 1595, 2023.

[9] N. F. Syed, S. W. Shah, A. Shaghaghi, A. Anwar, Z. Baig, and R. Doss, "Zero trust architecture (zta): A comprehensive survey," *IEEE Access*, vol. 10, pp. 57 143–57 179, 2022.

[10] V. A. Vora, "Demystifying zero trust security: The no-trust network paradigm," *Journal of Computer Science and Technology Studies*, vol. 7, no. 3, pp. 141–148, 2025.

[11] M. Tsai, S. Lee, and S. W. Shieh, "Strategy for implementing of zero trust architecture," *IEEE Transactions on Reliability*, vol. 73, no. 1, pp. 93–100, 2024.

[12] H. Bhoite, "Zero-trust architecture in streaming dataflows," https://www.authorea.com/doi/full/10.36227/techrxiv.175303721.12297807, accessed: 2026-03-26.

[13] "Broker configs," https://kafka.apache.org/42/configuration/broker-configs/, accessed: 2026-03-26.

[14] "Consumer configs," https://kafka.apache.org/42/configuration/consumer-configs/, accessed: 2026-03-26.

[15] J. Meka, "Financial services cloud transformation: Securing sensitive data in kafka event streams," *Journal of Computer Science and Technology Studies*, vol. 7, no. 4, pp. 1023–1028, 2025.

[16] A. A. Süzen and O. Ceylan, "The advantages and implementation challenges within the scope of the basic principles of transition to zero trust architecture," *International Journal of 3D Printing Technologies and Digital Industry*, vol. 8, no. 3, pp. 416–427, 2024.

[17] M. Mohammad, "Analysis of design patterns and benchmark practices in apache kafka event-streaming systems," in *2025 5th International Conference on Intelligent Cybernetics Technology & Applications (ICICyTA)*, 2025, pp. 612–617.

[18] C. Coarfa, P. Druschel, and D. S. Wallach, "Performance analysis of tls web servers," *ACM Transactions on Computer Systems*, vol. 24, no. 1, pp. 39–69, 2006.

[19] M. Iorio, F. Risso, and C. Casetti, "When latency matters: Measurements and lessons learned," *SIGCOMM Computer Communication Review*, vol. 51, no. 4, pp. 2–13, 2021.

[20] C. H. Jack, S. K. Teck, L. T. Ming, and D. Y. Hong, "An overview analysis of authentication mechanism in microservices-based software architecture: A discussion paper," in *2023 IEEE 8th International Conference on Software Engineering and Computer Systems (ICSECS)*, 2023, pp. 1–6.

[21] S. A. Jyothi, A. Singla, P. B. Godfrey, and A. Kolla, "Measuring and understanding throughput of network topologies," in *SC '16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2016, pp. 761–772.

Appendices

Appendix A

An Appendix of Some Kind

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. Aliquam lorem ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum. Aenean imperdiet. Etiam ultricies nisi vel augue. Curabitur ullamcorper ultricies nisi. Nam eget dui. Etiam rhoncus. Maecenas tempus, tellus eget condimentum rhoncus, sem quam semper libero, sit amet adipiscing sem neque sed ipsum. Nam quam nunc, blandit vel, luctus pulvinar, hendrerit id, lorem. Maecenas nec odio et ante tincidunt tempus. Donec vitae sapien ut libero venenatis faucibus. Nullam quis ante. Etiam sit amet orci eget eros faucibus tincidunt. Duis leo. Sed fringilla mauris sit amet nibh. Donec sodales sagittis magna. Sed consequat, leo eget bibendum sodales, augue velit cursus nunc.

Appendix B

Ignore - Templatey Stuff

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. [**?**] Aliquam lorem ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum.

$$M = \frac{1}{T}\sum_{t=1}^{T} e(t)/\max_{t}[e(t)] \tag{B.1}$$

Aenean imperdiet. Etiam ultricies nisi vel augue. Curabitur ullamcorper ultricies nisi. Nam eget dui. Etiam rhoncus. Maecenas tempus, tellus eget condimentum rhoncus, sem quam semper libero, sit amet adipiscing sem neque sed ipsum. Nam quam nunc, blandit vel, luctus pulvinar, hendrerit id, lorem. Maecenas nec odio et ante tincidunt tempus. Donec vitae sapien ut libero venenatis faucibus. Nullam quis ante. Etiam sit amet orci eget eros faucibus tincidunt. Duis leo. Sed fringilla mauris sit amet nibh. Donec sodales sagittis magna. Sed consequat, leo eget bibendum sodales, augue velit cursus nunc.

B.1 Section Template Graph

Content: Explain how microservices talk to each other.

Figure: Insert a diagram of a "Producer -¿ Broker -¿ Consumer" architecture here.

See Figure B.1.

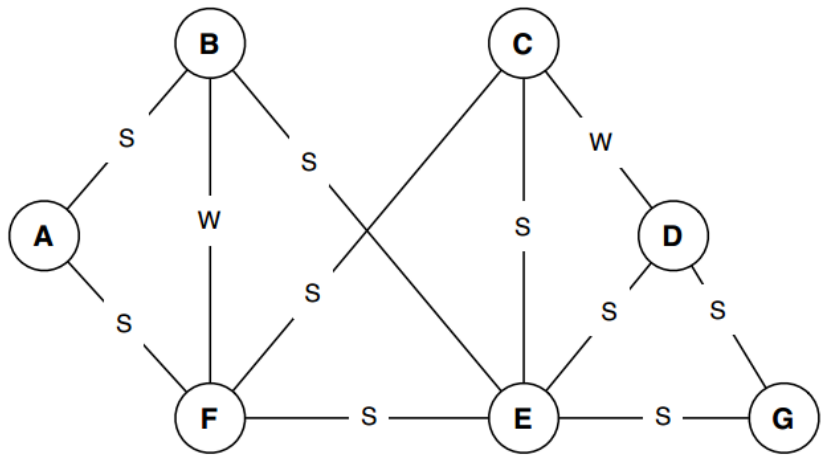

Figure B.1: A simple figure in LaTeX. Reproduced from http://tinyurl.com/nqtrlj5 with the permission of the copyright owner.

B.2 Section Template Comparison

Content: Lorem

Table: Use the template's table code to compare them (e.g., Column A=Lorem, B=Inps, C=Lode).

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

Table B.1: A simple table in LaTeX.

B.3 Summary

Content: "Existing benchmarks are outdated. This project fills that gap."

Appendix C

Another Appendix

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim. Donec pede justo, fringilla vel, aliquet nec, vulputate eget, arcu. In enim justo, rhoncus ut, imperdiet a, venenatis vitae, justo. Nullam dictum felis eu pede mollis pretium. Integer tincidunt. Cras dapibus. Vivamus elementum semper nisi. Aenean vulputate eleifend tellus. Aenean leo ligula, porttitor eu, consequat vitae, eleifend ac, enim. Aliquam lorem ante, dapibus in, viverra quis, feugiat a, tellus. Phasellus viverra nulla ut metus varius laoreet. Quisque rutrum. Aenean imperdiet. Etiam ultricies nisi vel augue. Curabitur ullamcorper ultricies nisi. Nam eget dui. Etiam rhoncus. Maecenas tempus, tellus eget condimentum rhoncus, sem quam semper libero, sit amet adipiscing sem neque sed ipsum. Nam quam nunc, blandit vel, luctus pulvinar, hendrerit id, lorem. Maecenas nec odio et ante tincidunt tempus. Donec vitae sapien ut libero venenatis faucibus. Nullam quis ante. Etiam sit amet orci eget eros faucibus tincidunt. Duis leo. Sed fringilla mauris sit amet nibh. Donec sodales sagittis magna. Sed consequat, leo eget bibendum sodales, augue velit cursus nunc.